Product Owner Document

# PO Hub Billing Provider

Hub de agrupamento de contas recorrentes de energia (CPFL / RGE), água (Aegea / Corsan) e telecom (Vivo) com envio ao sistema de aluguéis

| | |
|---|---|
| **Versão do documento** | **v1.7.0** |
| **Data** | 29 de outubro de 2026 |
| **Status** | Rascunho para validação |
| **Product Owner** | Weber |
| **Stack** | Python, RabbitMQ, Redis, Docker, Webhooks, API REST, testes unitários |
| **Plataformas** | Windows e Ubuntu (nativo e Docker) |
| **Artefatos desta versão** | PO_Hub_Billing_Provider_v1.7.0.pdf · Infografico_Hub_Billing_Provider_v1.7.0.pdf / .png |

# Sumário


1. Controle de versões
2. Visão do produto
3. Problema, objetivos e métricas de sucesso
4. Escopo
5. Atores e perfis de acesso
6. Provedores suportados
7. Requisitos funcionais
8. Requisitos não funcionais
9. Arquitetura
10. Filas, mensagens e Correlation ID
11. Padrão Adapter de provedores
12. Modelo de dados canônico
13. API REST e Webhooks
14. Segurança, usuários e permissões
15. Ambientes, Docker e execução em Windows/Ubuntu
16. Estratégia de testes
17. Política de versionamento do PO
18. Backlog e roadmap
19. Riscos, premissas e questões em aberto
20. Glossário


# 1. Controle de versões

| Versão | Data | Autor | Alterações |
|---|---|---|---|

| | | | |
|---|---|---|---|
| v1.7.0 | 2026-10-29 | Weber | • Versão automática após 14 PRs mergeados.<br>• PR #1: Adapter Vivo, novo PO a cada 10 PRs e PO v1.2.0<br>• PR #2: Importação de fatura em PDF e validação da linha digitável<br>• PR #3: Persistência em PostgreSQL: vínculos conta-imóvel, contas e lotes<br>• PR #4: Envio dos lotes ao sistema de aluguéis e workers de ponta a ponta<br>• PR #5: Coleta automática da RGE e da Corsan pelas APIs dos portais<br>• PR #6: Arquivos iniciar e aplicacao para Windows e Ubuntu<br>• PR #7: Painel de administração web em /painel<br>• PR #8: Cadastro do imóvel e do titular nas contas<br>• PR #9: Painel no estilo verde-petróleo, claro e escuro<br>• PR #10: Hub passa a usar a porta 8010 e avisa se a porta estiver ocupada<br>• PR #11: Usuários no banco e troca de senha obrigatória no painel<br>• PR #12: parar.bat e parar.sh para fechar o hub<br>• PR #13: iniciar.bat limpa sobras de instalações interrompidas<br>• PR #14: Conectar RGE e Corsan pelo painel |
| v1.6.0 | 2026-10-29 | Weber | • Versão automática após 13 PRs mergeados.<br>• PR #1: Adapter Vivo, novo PO a cada 10 PRs e PO v1.2.0<br>• PR #2: Importação de fatura em PDF e validação da linha digitável<br>• PR #3: Persistência em PostgreSQL: vínculos conta-imóvel, contas e lotes<br>• PR #4: Envio dos lotes ao sistema de aluguéis e workers de ponta a ponta<br>• PR #5: Coleta automática da RGE e da Corsan pelas APIs dos portais<br>• PR #6: Arquivos iniciar e aplicacao para Windows e Ubuntu<br>• PR #7: Painel de administração web em /painel<br>• PR #8: Cadastro do imóvel e do titular nas contas<br>• PR #9: Painel no estilo verde-petróleo, claro e escuro<br>• PR #10: Hub passa a usar a porta 8010 e avisa se a porta estiver ocupada<br>• PR #11: Usuários no banco e troca de senha obrigatória no painel<br>• PR #12: parar.bat e parar.sh para fechar o hub<br>• PR #13: iniciar.bat limpa sobras de instalações interrompidas |

| | | | |
|---|---|---|---|
| v1.5.0 | 2026-10-29 | Weber | • Versão automática após 12 PRs mergeados.<br>• PR #1: Adapter Vivo, novo PO a cada 10 PRs e PO v1.2.0<br>• PR #2: Importação de fatura em PDF e validação da linha digitável<br>• PR #3: Persistência em PostgreSQL: vínculos conta-imóvel, contas e lotes<br>• PR #4: Envio dos lotes ao sistema de aluguéis e workers de ponta a ponta<br>• PR #5: Coleta automática da RGE e da Corsan pelas APIs dos portais<br>• PR #6: Arquivos iniciar e aplicacao para Windows e Ubuntu<br>• PR #7: Painel de administração web em /painel<br>• PR #8: Cadastro do imóvel e do titular nas contas<br>• PR #9: Painel no estilo verde-petróleo, claro e escuro<br>• PR #10: Hub passa a usar a porta 8010 e avisa se a porta estiver ocupada<br>• PR #11: Usuários no banco e troca de senha obrigatória no painel<br>• PR #12: parar.bat e parar.sh para fechar o hub |
| v1.4.0 | 2026-10-29 | Weber | • Versão automática após 11 PRs mergeados.<br>• PR #1: Adapter Vivo, novo PO a cada 10 PRs e PO v1.2.0<br>• PR #2: Importação de fatura em PDF e validação da linha digitável<br>• PR #3: Persistência em PostgreSQL: vínculos conta-imóvel, contas e lotes<br>• PR #4: Envio dos lotes ao sistema de aluguéis e workers de ponta a ponta<br>• PR #5: Coleta automática da RGE e da Corsan pelas APIs dos portais<br>• PR #6: Arquivos iniciar e aplicacao para Windows e Ubuntu<br>• PR #7: Painel de administração web em /painel<br>• PR #8: Cadastro do imóvel e do titular nas contas<br>• PR #9: Painel no estilo verde-petróleo, claro e escuro<br>• PR #10: Hub passa a usar a porta 8010 e avisa se a porta estiver ocupada<br>• PR #11: Usuários no banco e troca de senha obrigatória no painel |

| | | | |
|---|---|---|---|
| v1.3.0 | 2026-10-29 | Weber | • Versão automática após 10 PRs mergeados.<br>• PR #1: Adapter Vivo, novo PO a cada 10 PRs e PO v1.2.0<br>• PR #2: Importação de fatura em PDF e validação da linha digitável<br>• PR #3: Persistência em PostgreSQL: vínculos conta-imóvel, contas e lotes<br>• PR #4: Envio dos lotes ao sistema de aluguéis e workers de ponta a ponta<br>• PR #5: Coleta automática da RGE e da Corsan pelas APIs dos portais<br>• PR #6: Arquivos iniciar e aplicacao para Windows e Ubuntu<br>• PR #7: Painel de administração web em /painel<br>• PR #8: Cadastro do imóvel e do titular nas contas<br>• PR #9: Painel no estilo verde-petróleo, claro e escuro<br>• PR #10: Hub passa a usar a porta 8010 e avisa se a porta estiver ocupada |
| v1.2.0 | 2026-10-28 | Weber | • Inclui a Vivo (telecom) como fornecedor de contas de consumo, com adapter vivo e tipo TELECOM.<br>• Regra de versionamento: novo PO (PDF + infográfico) a cada 10 PRs mergeados na main, gerado pela GitHub Action po-release.<br>• Testes ampliados para 63 (adapter Vivo e contagem de PRs). |
| v1.1.0 | 2026-10-25 | Weber | • Testes unitários ampliados (59 testes, cobertura 98%), incluindo RabbitMQ simulado, logs, usuários e gerador do PO.<br>• Testes executáveis em Docker (estágio test e serviço tests no Compose), Ubuntu e Windows, com scripts para cada ambiente.<br>• CI com testes no container, além da matriz ubuntu-latest e windows-latest. |
| v1.0.0 | 2026-10-25 | Weber | • Versão inicial do PO: visão, escopo, atores, provedores (CPFL/RGE, Aegea/Corsan), requisitos funcionais e não funcionais.<br>• Arquitetura com Python, RabbitMQ, Redis, PostgreSQL, Docker, adapters por provedor e Correlation ID.<br>• API REST, webhooks assinados (HMAC), conta admin e permissões por usuário.<br>• Backlog em 11 épicos, riscos, questões em aberto e política de versionamento (PDF + infográfico a cada alteração). |

## 2. Visão do produto

O **Hub Billing Provider** é um serviço central que coleta, normaliza e agrupa as contas recorrentes de consumo (energia elétrica, água e telecom) dos imóveis administrados, vincula cada conta ao

imóvel e ao contrato de locação correspondente e entrega essas contas, de forma confiável e rastreável, ao **sistema de aluguéis**.

O hub também expõe uma **API REST** e emite **webhooks** para que outros sistemas, acoplados no futuro, consumam os mesmos dados sem integração direta com as concessionárias.

> **Declaração de visão:** para administradores de imóveis que precisam repassar contas de consumo aos contratos de locação, o Hub Billing Provider é a fonte única e auditável de contas de energia, água e telecom, que elimina a coleta manual por concessionária e entrega as contas prontas para cobrança.

## 3. Problema, objetivos e métricas de sucesso

### 3.1 Problema

- Cada fornecedor (CPFL/RGE, Aegea/Corsan, Vivo) tem portal, formato e calendário próprios.
- A coleta manual é lenta, sujeita a erro e não deixa trilha de auditoria.
- O sistema de aluguéis precisa receber as contas já vinculadas ao imóvel/contrato, sem duplicidade.

### 3.2 Objetivos

- Automatizar a coleta de 100% das contas dos imóveis cadastrados nos provedores suportados.
- Agrupar as contas por imóvel, contrato e competência (mês de referência).
- Entregar as contas ao sistema de aluguéis com garantia de entrega e idempotência.
- Disponibilizar API e webhooks para novos sistemas consumidores.
- Permitir adicionar uma nova concessionária criando apenas um novo adapter.

### 3.3 Métricas de sucesso (KPIs)

| Métrica | Meta inicial |
|---|---|
| Contas coletadas automaticamente / contas esperadas no ciclo | ≥ 98% |
| Tempo entre disponibilidade da conta e entrega ao sistema de aluguéis | ≤ 24 h |
| Contas duplicadas entregues | 0 |
| Webhooks entregues com sucesso (após retentativas) | ≥ 99,5% |
| Cobertura de testes unitários | ≥ 80% |
| Tempo para incluir uma nova concessionária | ≤ 1 sprint |

## 4. Escopo

### Dentro do escopo (v1)

- Cadastro de imóveis, unidades consumidoras (energia), matrículas (água) e contas/linhas Vivo (telecom).
- Cadastro de credenciais de acesso por provedor, armazenadas cifradas.
- Coleta agendada e sob demanda via adapters CPFL/RGE, Aegea/Corsan e Vivo.
- Normalização para o modelo canônico de conta.
- Agrupamento por imóvel / contrato / competência.
- Envio ao sistema de aluguéis.
- API REST, webhooks assinados e painel de administração.
- Conta admin e usuários com permissões concedidas pelo admin.
- Logs estruturados com Correlation ID e auditoria.
- Docker Compose; execução em Windows e Ubuntu.

### Fora do escopo (v1)

- Pagamento das contas pelo hub.
- Cobrança ao inquilino (responsabilidade do sistema de aluguéis).
- Outras utilidades (gás, condomínio, outras operadoras de telecom), previstas para versões futuras.
- Aplicativo mobile.
- OCR avançado de contas digitalizadas em papel.

## 5. Atores e perfis de acesso

| Ator | Descrição |
|---|---|
| Administrador (Admin) | Conta raiz. Cria usuários, define perfis e permissões, gerencia provedores, credenciais, webhooks e chaves de API. |
| Usuário operador | Opera o dia a dia conforme permissões liberadas pelo admin: cadastra imóveis, acompanha coletas, reprocessa falhas. |
| Usuário consulta | Apenas visualiza contas, status e relatórios. |
| Sistema de aluguéis | Consumidor principal; recebe as contas agrupadas. |
| Sistemas acoplados | Consomem a API e/ou assinam webhooks com uma chave de API (client credentials). |
| Concessionárias | CPFL/RGE, Aegea/Corsan e Vivo: fontes das contas. |

## 6. Provedores suportados

| Provedor | Tipo | Identificador da conta | Observação |
|---|---|---|---|
| CPFL Paulista / Piratininga / Santa Cruz | Energia | Unidade Consumidora (UC) / Nº instalação | Adapter `cpfl` |
| RGE (grupo CPFL) | Energia | Unidade Consumidora (UC) | Adapter `cpfl` com distribuidora `rge` |
| Corsan (grupo Aegea) | Água / esgoto | Matrícula / Código do imóvel | Adapter `aegea` com unidade `corsan` |
| Demais unidades Aegea | Água / esgoto | Matrícula | Mesmo adapter, configurável por unidade |
| Vivo (Telefônica Brasil) | Telecom (fixo, internet, móvel) | Código do cliente / Nº da conta / linha | Adapter `vivo` |

> O meio de coleta de cada provedor (API oficial, portal do cliente, e-mail com fatura, arquivo EDI/CNAB ou upload manual) será confirmado na fase de descoberta (épico E1). O padrão adapter isola essa decisão do restante do hub.

## 7. Requisitos funcionais

| ID | Requisito | Prioridade |
|---|---|---|
| RF-01 | Autenticar usuários com login e senha; emitir token JWT com expiração e refresh. | **Must** |
| RF-02 | Existir uma conta admin inicial criada no primeiro boot (senha obrigatoriamente trocada no primeiro acesso). | **Must** |
| RF-03 | Admin cria, edita, bloqueia usuários e concede/revoga permissões (RBAC por perfil + permissões avulsas). | **Must** |
| RF-04 | CRUD de imóveis, com vínculo a contrato de locação (ID externo do sistema de aluguéis). | **Must** |
| RF-05 | CRUD de unidades consumidoras/matrículas por imóvel e por provedor. | **Must** |
| RF-06 | Cadastro de credenciais por provedor, armazenadas cifradas e nunca exibidas após salvas. | **Must** |
| RF-07 | Agendar coletas por provedor (cron configurável) e disparar coleta sob demanda via API/painel. | **Must** |
| RF-08 | Coletar as contas através de adapters por provedor, publicando cada resultado em fila. | **Must** |
| RF-09 | Normalizar cada conta para o modelo canônico (valor, vencimento, competência, código de barras, linha digitável, PIX, PDF). | **Must** |
| RF-10 | Detectar duplicidade (mesmo provedor + identificador + competência) e não reprocessar. | **Must** |

| | | |
|---|---|---|
| RF-11 | Agrupar as contas por imóvel / contrato / competência, gerando um "lote de cobrança". | **Must** |
| RF-12 | Enviar o lote ao sistema de aluguéis, com confirmação (ack) e retentativas. | **Must** |
| RF-13 | Disponibilizar API REST de leitura de contas, lotes, imóveis e status de coleta. | **Must** |
| RF-14 | Permitir que sistemas externos registrem assinaturas de webhook por tipo de evento. | **Must** |
| RF-15 | Enviar webhooks assinados (HMAC-SHA256) com retentativa exponencial e DLQ. | **Must** |
| RF-16 | Propagar Correlation ID em toda requisição, mensagem, log e webhook. | **Must** |
| RF-17 | Painel admin: status das filas, coletas, falhas e botão de reprocessar. | **Should** |
| RF-18 | Registrar trilha de auditoria (quem, o quê, quando, Correlation ID). | **Should** |
| RF-19 | Alertar (e-mail/webhook) quando conta esperada não for encontrada até a data limite. | **Should** |
| RF-20 | Upload manual de fatura (PDF) quando a coleta automática não for possível. | **Should** |
| RF-21 | Relatórios de consumo histórico por imóvel (kWh, m³, valores). | **Could** |

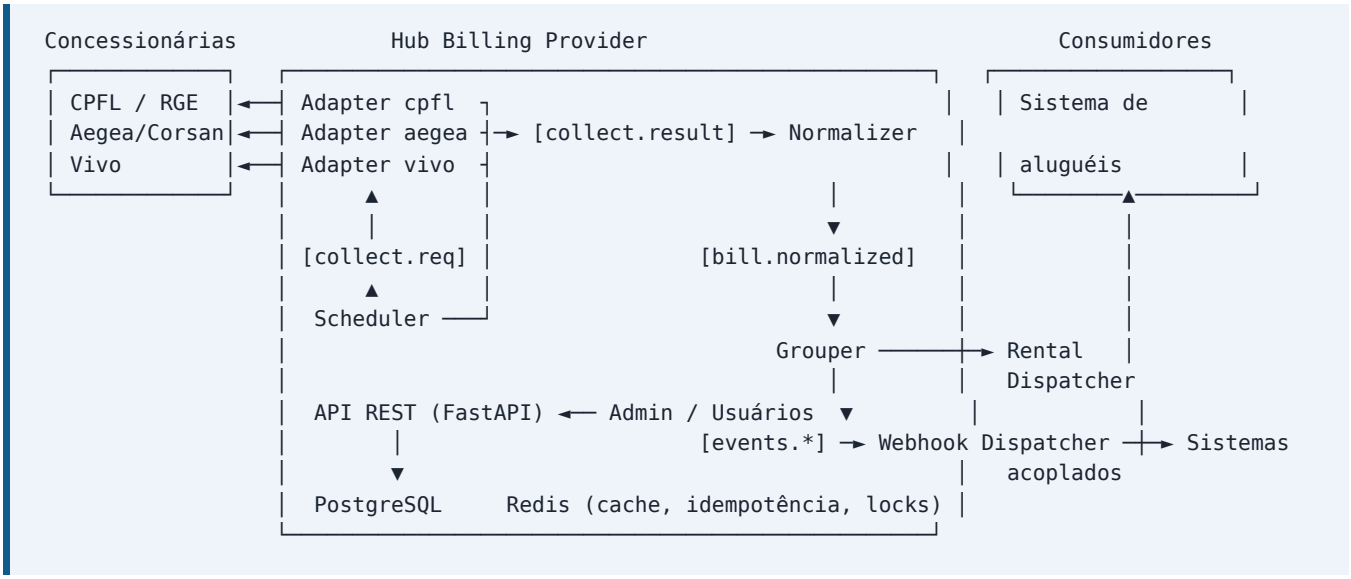
## 8. Requisitos não funcionais

| ID | Categoria | Requisito |
|---|---|---|
| RNF-01 | Linguagem | Python 3.12+, tipagem estática (mypy), lint com ruff. |
| RNF-02 | Portabilidade | Executar em Windows 10/11 e Ubuntu 22.04/24.04, nativamente e via Docker. |
| RNF-03 | Containers | Docker e Docker Compose para todos os serviços (api, workers, scheduler, rabbitmq, redis, postgres). |
| RNF-04 | Mensageria | RabbitMQ com filas duráveis, mensagens persistentes, ack manual, DLQ e filas de retry com TTL. |
| RNF-05 | Cache e controle | Redis para cache, chaves de idempotência, locks distribuídos e rate limit por provedor. |
| RNF-06 | Rastreabilidade | Correlation ID (UUID v4) em logs JSON estruturados; cabeçalho `X-Correlation-ID`. |
| RNF-07 | Confiabilidade | Entrega "at-least-once" com consumidores idempotentes; nenhuma conta perdida em caso de queda. |
| RNF-08 | Segurança | Senhas com Argon2/bcrypt; credenciais de provedores cifradas (Fernet/AES-256); TLS em produção; segredos via variáveis de ambiente. |
| RNF-09 | LGPD | Dados pessoais minimizados, mascarados em logs; política de retenção configurável. |

| | | |
|---|---|---|
| RNF-10 | Testes | pytest com cobertura ≥ 80%; adapters testados com respostas gravadas (fixtures); testes executáveis em Ubuntu, Windows e Docker. |
| RNF-11 | Desempenho | API com p95 < 300 ms em leituras; coleta paralela configurável por provedor. |
| RNF-12 | Observabilidade | Health checks (`/health`, `/ready`), métricas Prometheus (opcional na v1). |
| RNF-13 | Documentação | OpenAPI gerado automaticamente; PO versionado em PDF + infográfico a cada alteração. |

# 9. Arquitetura

## 9.1 Visão de componentes




## 9.2 Componentes

| Componente | Responsabilidade | Tecnologia proposta |
|---|---|---|
| API | Autenticação, RBAC, CRUDs, consulta de contas, gestão de webhooks, disparo de coleta. | FastAPI + Uvicorn |
| Scheduler | Publica pedidos de coleta conforme o calendário de cada provedor. | APScheduler |
| Collector workers | Consomem `collect.request`, chamam o adapter do provedor, publicam o resultado. | Python + aio-pika |
| Normalizer | Converte dados brutos em conta canônica; valida e deduplica. | Pydantic v2 |
| Grouper | Agrupa contas por imóvel/contrato/competência e fecha lotes. | Python |
| Rental Dispatcher | Entrega lotes ao sistema de aluguéis e registra ack. | httpx |
| Webhook Dispatcher | Entrega eventos aos assinantes com HMAC, retry e DLQ. | httpx |
| Banco de dados | Persistência de usuários, imóveis, contas, lotes, auditoria. | PostgreSQL 16 + SQLAlchemy 2 + Alembic |
| Redis | Cache, idempotência, locks, rate limit, sessões de provedor. | Redis 7 |
| RabbitMQ | Barramento de mensagens entre todos os componentes. | RabbitMQ 3.13 (management) |

> PostgreSQL é uma proposta deste PO (o pedido original não definiu banco). Pode ser trocado por

> outro banco relacional sem impacto na arquitetura.

### 9.3 Estrutura de pastas proposta

```
hub-billing-provider/
├─ src/hub_billing/
│  ├─ api/            # rotas FastAPI, auth, RBAC
│  ├─ core/           # config, logging, correlation id, segurança
│  ├─ domain/         # modelos canônicos e regras de agrupamento
│  ├─ adapters/
│  │  ├─ base.py      # interface ProviderAdapter
│  │  ├─ cpfl/        # CPFL + RGE
│  │  ├─ aegea/       # Aegea + Corsan
│  │  └─ vivo/        # Vivo (telecom)
│  ├─ messaging/      # publisher/consumer RabbitMQ, topologia
│  ├─ workers/        # collector, normalizer, grouper, dispatchers
│  ├─ webhooks/       # assinaturas, assinatura HMAC, entrega
│  └─ infra/          # db, redis, repositórios
├─ tests/unit/ · tests/integration/
├─ docs/po/           # VERSION, CHANGELOG, templates, vX.Y.Z/ (PDF + infográfico)
├─ tools/build_po.py  # gera o PDF e o infográfico versionados
├─ docker/ · docker-compose.yml · Makefile · scripts/*.ps1
└─ pyproject.toml
```

## 10. Filas, mensagens e Correlation ID

### 10.1 Topologia RabbitMQ

| Exchange (tipo) | Routing key / Fila | Produtor → Consumidor |
|---|---|---|
| `hub.collect` (topic) | `collect.request.<provider>` | Scheduler/API → Collector |
| `hub.collect` (topic) | `collect.result.<provider>` | Collector → Normalizer |
| `hub.bills` (topic) | `bill.normalized` | Normalizer → Grouper |
| `hub.bills` (topic) | `batch.ready` | Grouper → Rental Dispatcher |
| `hub.events` (fanout) | `events.webhook` | Todos → Webhook Dispatcher |
| `hub.retry` (direct) | `*.retry.{30s,5m,1h}` (TTL + dead-letter de volta) | Retentativas com backoff |
| `hub.dlx` (direct) | `*.dlq` | Falhas definitivas → painel admin |

### 10.2 Envelope de mensagem

```
{
  "message_id": "7d1f...-uuid",          // único por mensagem (idempotência)
  "correlation_id": "c0a8...-uuid",      // único por fluxo de negócio
  "causation_id": "uuid-da-mensagem-que-originou",
  "type": "bill.normalized",
  "version": 1,
  "occurred_at": "2026-10-25T03:00:00Z",
  "producer": "normalizer",
  "attempt": 1,
  "payload": { ... }
}
```

### 10.3 Regras do Correlation ID

- Gerado na entrada do fluxo (API, scheduler ou upload) se não vier no cabeçalho `X-Correlation-ID`.
- Copiado para a propriedade AMQP `correlation_id` e para o envelope de todas as mensagens derivadas.
- Incluído em todos os logs (via `contextvars`), registros de auditoria, chamadas ao sistema de aluguéis e webhooks.
- Retornado no cabeçalho de resposta da API, permitindo rastrear uma conta de ponta a ponta.

### 10.4 Idempotência

- Chave de negócio da conta: `provider + account_id + reference_month` (único no banco).
- Chave de mensagem: `message_id` gravado no Redis com TTL; mensagens repetidas recebem ack sem reprocessar.
- Envio ao sistema de aluguéis usa cabeçalho `Idempotency-Key` = ID do lote.

## 11. Padrão Adapter de provedores

Cada concessionária é implementada como um adapter que obedece a uma interface única. O restante do hub nunca conhece detalhes do provedor. Adapters são registrados por nome e habilitados por configuração.

```
class ProviderAdapter(Protocol):
    code: str                       # "cpfl", "aegea", "vivo"
    utility_type: UtilityType       # ENERGY | WATER | TELECOM

    async def authenticate(self, credentials: ProviderCredentials) -> Session: ...
    async def list_accounts(self, session: Session) -> list[ProviderAccount]: ...
    async def fetch_bills(self, session: Session, account: ProviderAccount,
                          reference: YearMonth | None = None) -> list[RawBill]: ...
    async def download_document(self, session: Session, bill: RawBill) -> bytes: ...
    def to_canonical(self, raw: RawBill) -> Bill: ...
```

- **Registro:** `AdapterRegistry.register("cpfl", CpflAdapter)`; novos provedores entram sem alterar o núcleo.
- **Variações:** distribuidoras/unidades (ex.: RGE, Corsan) são parâmetros do adapter, não novos adapters.
- **Resiliência:** timeout, retry com backoff, circuit breaker e rate limit (Redis) por provedor.

- **Testabilidade:** cada adapter possui um `FakeAdapter` e fixtures de respostas reais anonimizadas.

## 12. Modelo de dados canônico

| Entidade | Campos principais |
|---|---|
| User | id, nome, e-mail, hash_senha, ativo, is_admin, perfis[], permissões[], criado_em |
| Property (imóvel) | id, código, endereço, contrato_externo_id, ativo |
| UtilityAccount | id, property_id, provider, distribuidora/unidade, account_number (UC/matrícula), titular, ativo |
| ProviderCredential | id, provider, login, segredo_cifrado, escopo, último_sucesso |
| Bill (conta) | id, utility_account_id, provider, utility_type, reference_month, issue_date, due_date, amount, consumo (kWh/m³), barcode, digitable_line, pix_code, document_url, status, correlation_id |
| BillingBatch (lote) | id, property_id, contrato_externo_id, reference_month, bills[], total, status (OPEN, READY, SENT, ACKED, FAILED), sent_at |
| WebhookSubscription | id, client_id, url, eventos[], segredo, ativo |
| WebhookDelivery | id, subscription_id, event_id, tentativa, status_http, próxima_tentativa |
| AuditLog | id, actor, ação, recurso, antes/depois, correlation_id, timestamp |

**Status da conta:** `COLLECTED → NORMALIZED → GROUPED → SENT → ACKNOWLEDGED`, com desvios `DUPLICATE`, `FAILED` e `MANUAL_REVIEW`.

## 13. API REST e Webhooks

### 13.1 Endpoints principais (prefixo `/api/v1`)

| Método | Rota | Descrição |
|---|---|---|
| POST | /auth/login · /auth/refresh | Autenticação de usuário (JWT) |
| POST | /auth/token | Client credentials para sistemas acoplados |
| CRUD | /users · /roles · /permissions | Gestão de acesso (admin) |
| CRUD | /properties · /utility-accounts | Imóveis e UCs/matrículas |
| CRUD | /providers/{code}/credentials | Credenciais de provedores |
| POST | /collections | Dispara coleta sob demanda (retorna correlation_id) |
| GET | /collections/{id} | Status da coleta |
| GET | /bills · /bills/{id} · /bills/{id}/document | Consulta de contas e PDF |
| GET | /batches · /batches/{id} | Lotes enviados ao sistema de aluguéis |

| | | |
|---|---|---|
| POST | /batches/{id}/resend | Reenvio manual |
| CRUD | /webhooks | Assinaturas de webhook |
| GET | /health · /ready | Saúde do serviço |

### 13.2 Eventos de webhook

| Evento | Quando |
|---|---|
| `bill.collected` | Nova conta coletada e normalizada |
| `bill.updated` | Conta alterada (ex.: segunda via, novo valor) |
| `bill.missing` | Conta esperada não encontrada até a data limite |
| `batch.sent` / `batch.acknowledged` / `batch.failed` | Ciclo de vida do lote no sistema de aluguéis |
| `collection.failed` | Falha definitiva de coleta em um provedor |

### 13.3 Contrato de entrega

- Cabeçalhos: `X-Hub-Event`, `X-Hub-Delivery`, `X-Correlation-ID`, `X-Hub-Timestamp`, `X-Hub-Signature-256` (HMAC-SHA256 de `timestamp.body`).
- Sucesso = HTTP 2xx em até 10 s. Caso contrário, retentativas em 30 s, 5 min, 1 h, 6 h e 24 h; depois DLQ e alerta.
- Consumidores devem ser idempotentes usando `X-Hub-Delivery`.

### 13.4 Integração com o sistema de aluguéis

Modo padrão: o hub faz `POST` do lote para a API do sistema de aluguéis com `Idempotency-Key`. Modo alternativo: o sistema de aluguéis assina o webhook `batch.ready` ou consulta `GET /batches`. O contrato exato será definido com o time do sistema de aluguéis (questão em aberto Q-01).

## 14. Segurança, usuários e permissões

Controle de acesso baseado em perfis (RBAC) com permissões granulares. A conta **admin** possui todas as permissões e é a única que pode conceder permissões a outros usuários.

| Permissão | Admin | Operador | Consulta |
|---|---|---|---|
| users:manage, roles:manage | ✔ | – | – |
| providers:credentials:write | ✔ | conforme admin | – |
| properties:write, accounts:write | ✔ | ✔ | – |
| collections:run, batches:resend | ✔ | ✔ | – |
| bills:read, batches:read | ✔ | ✔ | ✔ |
| webhooks:manage, api-clients:manage | ✔ | conforme admin | – |
| audit:read | ✔ | conforme admin | – |

- Toda ação de escrita é registrada na auditoria com o Correlation ID.
- Bloqueio após tentativas de login inválidas (rate limit no Redis).
- Chaves de API de sistemas acoplados com escopos e expiração; rotação pelo admin.

## 15. Ambientes, Docker e execução em Windows/Ubuntu

| Serviço (docker-compose) | Imagem | Porta |
|---|---|---|
| api | hub-billing (python:3.12-slim) | 8000 |
| scheduler · worker-collector · worker-normalizer · worker-grouper · worker-dispatcher · worker-webhooks | hub-billing | – |
| rabbitmq | rabbitmq:3.13-management | 5672 / 15672 |
| redis | redis:7-alpine | 6379 |
| postgres | postgres:16-alpine | 5432 |

- **Ubuntu:** `make up` / `make test`; execução nativa com `python -m venv`.
- **Windows:** Docker Desktop (WSL2) ou nativo com scripts PowerShell equivalentes (`scripts\up.ps1`, `scripts\test.ps1`).
- Código sem dependência de SO: `pathlib`, sem chamadas de shell, encoding UTF-8 explícito; CI testa nas duas plataformas.
- Configuração por variáveis de ambiente (`.env`), via pydantic-settings.

## 16. Estratégia de testes

| Nível | Escopo | Ferramentas |
|---|---|---|
| Unitário | Domínio, agrupamento, normalização, adapters (com fixtures), RabbitMQ com canal simulado (retry/DLQ), idempotência (fakeredis), assinatura HMAC, API e RBAC, logs/Correlation ID, gerador do PO | pytest, pytest-asyncio, respx, fakeredis, unittest.mock |
| Integração | RabbitMQ, Redis e Postgres reais | testcontainers / docker compose |
| Contrato | API OpenAPI e payloads de webhook/lote | schemathesis |
| CI | Lint, mypy, testes e cobertura em ubuntu-latest e windows-latest (Python 3.12 e 3.13) e dentro do container de teste | GitHub Actions |

### 16.1 Onde os testes unitários rodam

Os testes unitários não dependem de RabbitMQ, Redis ou PostgreSQL rodando (usam simulações), então executam igual nos três ambientes. Todos rodam ruff, mypy e pytest com cobertura mínima de 80%.

| Ambiente | Comando |
|---|---|
| Ubuntu | `make install` e `make test` |
| Windows (PowerShell) | `.\scripts\install.ps1` e `.\scripts\test.ps1` |
| Docker (Ubuntu ou Windows) | `make test-docker`, `./scripts/test-docker.sh` ou `.\scripts\test-docker.ps1` (estágio `test` do Dockerfile, serviço `tests` do Compose) |

Situação na v1.7.0: 63 testes unitários, cobertura de 98%.

Critério de pronto (DoD): código revisado, testes unitários passando, cobertura ≥ 80% no módulo alterado, documentação atualizada e, quando houver mudança de escopo/requisito, nova versão do PO gerada.

## 17. Política de versionamento do PO

- Versionamento semântico do documento: **MAJOR.MINOR.PATCH**.
  - **MAJOR**: mudança de visão, escopo ou arquitetura que quebra o acordado.
  - **MINOR**: novo requisito, provedor, evento ou épico.
  - **PATCH**: correções de texto, esclarecimentos, ajustes de prioridade.
- A cada **10 PRs mergeados** na `main` é gerado automaticamente um **novo PDF e um novo infográfico** (versão MINOR), com os títulos dos PRs no changelog: `docs/po/vX.Y.Z/PO_Hub_Billing_Provider_vX.Y.Z.pdf` e `Infografico_Hub_Billing_Provider_vX.Y.Z.(pdf|png)`. Versões anteriores nunca são sobrescritas.
- Automação: GitHub Action `po-release` executa `python tools/build_po.py --every-prs 10 --git` a cada push na `main`; o contador zera a cada nova tag `po-vX.Y.Z`.
- Mudanças de visão/escopo (MAJOR) ou urgentes podem gerar versão manual a qualquer momento com `--bump`.
- O arquivo `docs/po/VERSION` guarda a versão atual e `docs/po/CHANGELOG.md` o histórico (também impresso na seção 1).
- Geração: `python tools/build_po.py --bump minor -m "descrição"` (funciona em Windows e Ubuntu).
- Os artefatos são salvos no PC (pasta local do projeto) e no GitHub (commit + tag `po-vX.Y.Z`).

## 18. Backlog e roadmap

| Épico | Nome | Histórias principais | Sprint |
|---|---|---|---|
| E0 | Fundação | Repositório, pyproject, lint/mypy, Docker Compose, CI Windows+Ubuntu, logging JSON com Correlation ID, gerador do PO. | 1 |
| E1 | Descoberta de provedores | Levantar meio de coleta CPFL/RGE, Aegea/Corsan e Vivo (API, portal, e-mail, EDI); anonimizar amostras de faturas. | 1 |
| E2 | Acesso e usuários | Conta admin inicial, login JWT, CRUD de usuários, perfis e permissões, auditoria. | 2 |
| E3 | Cadastros | Imóveis, contratos, UCs/matrículas, credenciais cifradas. | 2 |
| E4 | Mensageria | Topologia RabbitMQ, envelope, retry/DLQ, idempotência no Redis. | 3 |
| E5 | Adapter CPFL/RGE | Autenticação, listagem de UCs, coleta, download do PDF, normalização, testes. | 3–4 |
| E6 | Adapter Aegea/Corsan | Idem para matrículas de água. | 4–5 |
| E6b | Adapter Vivo | Idem para contas de telecom (código do cliente / linha). | 5 |
| E7 | Agrupamento e envio | Grouper, lotes, Rental Dispatcher, ack e reenvio. | 5 |
| E8 | API pública e webhooks | Endpoints de leitura, client credentials, assinaturas, HMAC, entregas, DLQ. | 6 |
| E9 | Painel admin e operação | Dashboard de filas/coletas, reprocessamento, alertas de conta ausente, upload manual. | 7 |
| E10 | Go-live | Hardening, LGPD, backup, documentação de implantação, piloto com imóveis reais. | 8 |

### Exemplos de histórias (formato de aceite)

| ID | História | Critérios de aceite |
|---|---|---|
| US-01 | Como admin, quero conceder permissões a um usuário para que ele opere apenas o necessário. | Permissão concedida reflete no próximo token; ação registrada na auditoria; usuário sem permissão recebe 403. |
| US-02 | Como operador, quero disparar a coleta de um imóvel e acompanhar pelo Correlation ID. | POST /collections retorna 202 + correlation_id; GET mostra o status de cada etapa. |
| US-03 | Como sistema de aluguéis, quero receber cada conta uma única vez por competência. | Reenvio da mesma conta não cria duplicidade; Idempotency-Key presente; ack registrado. |
| US-04 | Como sistema acoplado, quero assinar `bill.collected` e validar a assinatura. | Webhook entregue com HMAC válido; falha gera retentativas; após esgotar vai para DLQ. |
| US-05 | Como dev, quero adicionar um novo provedor implementando só um adapter. | Novo adapter registrado e coberto por testes, sem mudança fora de `adapters/`. |

# 19. Riscos, premissas e questões em aberto

## 19.1 Riscos

| Risco | Impacto | Mitigação |
|---|---|---|
| Concessionária sem API oficial; portal muda layout ou usa captcha | Alto | Adapter isolado, testes com fixtures, alerta de falha, upload manual como contingência, buscar convênio/EDI. |
| Termos de uso dos portais restringem automação | Alto | Validação jurídica na E1; priorizar canais oficiais (API, e-mail de fatura, EDI). |
| Credenciais de clientes vazadas | Alto | Cifragem, segredos fora do código, acesso restrito por permissão, auditoria. |
| Contrato da API do sistema de aluguéis indefinido | Médio | Dispatcher com adapter próprio; modo webhook/pull como alternativa. |
| Diferenças Windows × Ubuntu | Baixo | Docker como padrão, CI nas duas plataformas. |

## 19.2 Premissas

- Os imóveis e contratos já existem no sistema de aluguéis e têm um ID externo estável.
- O titular das contas autoriza o acesso às faturas (LGPD).

## 19.3 Questões em aberto

| ID | Questão |
|---|---|
| Q-01 | Qual é o contrato (API/formato) do sistema de aluguéis para receber as contas? |
| Q-02 | Qual o meio de coleta disponível em cada concessionária (API, portal, e-mail, EDI)? |
| Q-03 | Quantos imóveis/UCs na largada e qual o crescimento esperado? |
| Q-04 | O painel admin será web (no próprio hub) ou parte de outro sistema? |
| Q-05 | Onde o hub será hospedado em produção (servidor Windows, Ubuntu, nuvem)? |
| Q-06 | Confirmação do PostgreSQL como banco de dados. |

# 20. Glossário

| Termo | Significado |
|---|---|
| UC | Unidade Consumidora de energia elétrica. |
| Matrícula | Identificador do imóvel na concessionária de água. |
| Código do cliente (Vivo) | Identificador da conta de telecom na Vivo. |
| Competência | Mês de referência da conta (AAAA-MM). |
| Adapter | Componente que traduz um provedor específico para a interface padrão do hub. |

| | |
|---|---|
| Correlation ID | Identificador único que acompanha um fluxo de negócio em todas as etapas. |
| DLQ | Dead Letter Queue: fila de mensagens que falharam definitivamente. |
| Idempotência | Processar a mesma mensagem mais de uma vez sem efeito duplicado. |
| Webhook | Chamada HTTP enviada pelo hub a um sistema externo quando ocorre um evento. |
| Lote (batch) | Conjunto de contas de um imóvel/contrato em uma competência. |